

Module III

Open Stack Cluster – The Cloud Controller and Common Services- Asymmetric clustering, Symmetric clustering, The cloud controller - The keystone service.

Open Stack Clusters

- A cluster is a group of homogeneous objects (i.e. [Node](#)).
 - A [Node](#) is an object that belongs to at most one [Cluster](#).
 - A node can become an ‘orphaned node’ when it is not a member of any clusters.
 - All nodes in a cluster must be of the same [Profile Type](#) of the owning cluster.
 - In general, a node represents a physical object exposed by other OpenStack services.
 - A node has a unique [Index](#) value scoped to the cluster that owns it.
 - An [Index](#) value is an integer property of a [Node](#) when it is a member of a [Cluster](#). Each node has an auto-generated index value that is unique in the cluster.
- In Open Stack, clusters refer to groups of logical objects managed by the **Senlin service**.
- These objects, called nodes, can be various resources like compute instances, bare-metal servers, or containers.
- **Senlin** offers APIs and command-line tools for creating, managing, and scaling clusters.
- A cluster consists of 0 or more nodes and it can be associated with 0 or more [Policy](#) objects.
 - A [Policy](#) is a set of rules that can be checked and/or enforced when an [Action](#) is performed on a [Cluster](#).
 - An [Action](#) is an operation that can be performed on a [Cluster](#) or a [Node](#) etc.
 - Different types of objects support different set of actions.
 - An action is executed by a [Worker](#) thread when the action becomes READY.
 - A [Worker](#) is the thread created and managed by Senlin engine to execute an [Action](#) that becomes ready. When the current action completes (with a success or failure), a worker will check the database to find another action for execution.

- Most Senlin APIs create actions in database for worker threads to execute asynchronously.
- An action, when executed, will check and enforce [Policy](#) associated with the cluster.
- An action can be triggered via [Receiver](#).
- A [Receiver](#) is an abstract resource created at the senlin engine that can be used to hook the engine to some external event/alarm sources. A receiver can be of different types. The most common type is a [Webhook](#).
- A [Webhook](#) is an encoded URI (Uniform Resource Identifier) that for triggering some operations (e.g. Senlin actions) on some resources. Such a webhook URL is the only thing one needs to know to trigger an action on a cluster.
- A policy is an instance of a particular [Policy Type](#).
 - A policy type is an abstraction of [Policy](#) objects.
 - The implementation of a policy type specifies when the policy should be checked and/or enforce, what profile types are supported, what operations are to be done before, during and after each [Action](#).
 - All policy types are provided as Senlin plugins.
- Users can specify the enforcement level when creating a policy object.
- Such a policy object can be attached to and detached from a cluster.
- It is associated with a [Profile Type](#) when created.
 - A [Profile Type](#) is an abstraction of objects that are backed by some [Driver](#)
 - A [Driver](#) is a Senlin internal module that enables Senlin [Engine](#) to interact with other [OpenStack](#) services.
 - Senlin [Engine](#) is the daemon that actually performs the operations requested by users. It provides RPC interfaces to RPC clients.
 - The interactions here are usually used to create, destroy, update the objects exposed by those services.
 - The implementation of a profile type calls the driver(s) to create objects that are managed by Senlin.
 - The implementation also serves a factory that can “produce” objects given a profile. All profile types are provided as Senlin plugins.

Clustering Approaches:

Asymmetric Clustering vs. Symmetric Clustering

Asymmetric Clustering: Centralized control with a master node (cloud controller) managing other nodes. Simpler but creates a single point of failure.

- This approach involves a designated master node (cloud controller)
- It manages and coordinates the other nodes (compute nodes).
- The master node has additional responsibilities like scheduling tasks, handling authentication, and providing a single point of access.
- This setup is often preferred for its simplicity and centralized control, but it creates a single point of failure if the master node goes down.

Symmetric Clustering: Distributed approach with all nodes as peers. More fault-tolerant and scalable but complex to manage.

- Here, all nodes are peers with equal capabilities.
- There is no designated master, and tasks are distributed among nodes for better fault tolerance and scalability.
- However, this structure can be more complex to manage and requires additional mechanisms for coordination and consensus among the nodes.

The Cloud Controller (Keystone Service)

- Central authority for authentication and authorization.
- The cloud controller, often implemented using Keystone, is a crucial component in OpenStack, especially in asymmetric clusters.
- Users/applications authenticate with Keystone to access resources.
- Manages user accounts, tenants (projects), roles, and tokens
- It acts as the central authority for authentication and authorization.

- Users and applications must authenticate with Keystone to access OpenStack resources.
- Keystone manages:
 - User Accounts: Stores user information and credentials.
 - Tenants (Projects): Groups users with shared access rights.
 - Roles: Defines permissions that users or projects can have.
 - Tokens: Provides temporary access credentials for authenticated users.

Common Services in OpenStack Clusters

Beyond Keystone, various services provide essential functionalities within an OpenStack cluster:

- **Nova (Compute Service):** Manages the lifecycle of virtual machines (VMs).
- **Neutron (Networking Service):** Creates and configures virtual networks for VMs.
- **Cinder (Block Storage Service):** Provides persistent block storage for VMs.
- **Glance (Image Service):** Stores and manages VM images.
- **Swift (Object Storage Service):** Offers scalable object storage for unstructured data.
- **Heat (Orchestration Service):** Automates the deployment and management of cloud resources.

Additional Considerations

- The specific services and their role in the cloud controller might vary depending on the **Open Stack** deployment and chosen clustering approach.
- Some services can be deployed in either an asymmetric or symmetric fashion. For example, Nova can have a centralized scheduling component or a distributed one.
- Security is paramount when managing **OpenStack clusters**. Secure communication between components and proper role-based access control are essential.